Fedlify: governed preparation for multi-institutional health AI studies, with bounded agentic assistance

Omid Jafarinezhad[1], Mohammad Noaeen[1], Aviv Shachak[1,3], Behrouz Far[4], and Zahra Shakeri[1,3,5,*]

[1]Institute of Health Policy, Management and Evaluation, Dalla Lana School of Public Health, University of Toronto, Toronto, Canada
[3]Faculty of Information, University of Toronto, Toronto, Canada
[4]Department of Electrical and Software Engineering, University of Calgary, Calgary, Canada
[5]Schwartz Reisman Institute, University of Toronto, Toronto, Canada
[*]Corresponding author, zahra.shakeri@utoronto.ca

context, gap, Fedlify, approach, key results, and impact.
[valley of death → poor generalizability → need for multi-institutional data → federated learning → operational complexity → limited adoption → need for better operational tools → agentic AI for planning and preparation → bounded autonomy with human oversight → Fedlify.]
In a six-hospital eICU heart-failure mortality case study, Fedlify transformed an incomplete study description into a confirmed specification, identified a missing subgroup-evaluation plan, coordinated site-specific governance and readiness records, generated five verified NVIDIA FLARE jobs, and produced credential-free startup packages for local deployment. The case study demonstrates technical feasibility and a traceable separation between preparation and execution.

Introduction

A persistent challenge in health AI is the "valley of death" between developing a technically promising model and establishing its safe, effective and sustainable use in routine clinical care.[1–3] Recent guidance based on the deployment of AI agents in clinical practice quantifies this gap, and reports that prompt engineering and model development accounted for less than 20% of the effort, while data integration, validation, economic value assessment, drift management, and governance accounted for more than 80%.[2] Bridging this gap requires a sociotechnical approach that considers clinical workflows, organizational structures, governance, human factors, and technical infrastructure alongside algorithmic performance.[4,5] One of the first challenges in crossing this valley is limited generalizability, which often arises when development data do not adequately represent the target population or the clinical setting.

Performance at the development site does not establish performance elsewhere. For example, COVID-19 screening models trained at Oxford University Hospitals achieved AUROCs of 0.757–0.817 in four UK cohorts, but only 0.577 and 0.515 at two Vietnamese hospitals, which were close to random discrimination.[6] One source of this failure is shortcut learning, in which models rely on hidden data-acquisition cues rather than clinically meaningful patterns. Across 13 clinical datasets and five modalities, these shortcuts caused internal evaluations to overestimate external performance by 20% on average.[7] Underrepresentation compounds this problem because models can become less robust and less fair when patient groups, conditions, or acquisition settings encountered during deployment were poorly represented during development.[8]

Institutional diversity can substantially improve model performance and generalizability, but its practical realization is still challenging. Recent large-scale health AI collaborations quantify these improvements. A glioblastoma model trained on 25,256 MRI scans from 6,314 patients at 71 institutions on 6 continents improved delineation of the surgically targetable tumour by 33% and the complete tumour extent by 23% compared with a model trained on public data.[9] A COVID-19 model trained across 20 institutions achieved an average AUC above 0.92, 16% higher than single-site models, with 38% better generalizability.[10] A paediatric brain tumour model developed with 19 international sites similarly improved segmentation by 20–30% at three external hospitals while performing close to centralized training.[11] However, in practice, such collaboration is constrained by privacy regulations, governance requirements, and institutional barriers surrounding the sharing of sensitive health data. These challenges are further amplified in countries with decentralized healthcare governance, such as Canada, Germany, and Australia, where jurisdiction-specific regulatory frameworks and fragmented data infrastructures add layers of complexity.[12,13] As a result, many health AI models are trained on single-institution datasets, which constrains their generalizability and clinical reliability in diverse patient populations.[14–17]

The need for institutional diversity does not imply that health data must be centralized. Transfer learning adapts a model sequentially by using local data to refine a model trained elsewhere, as when the Oxford COVID-19 models were fine-tuned on Vietnamese data.[6] Federated learning (FL) instead enables institutions to train collaboratively and build a shared

model without transferring raw patient data.[18,19] Federated transfer learning combines both principles. However, moving from a technical demonstration to a practical multi-institutional study is difficult. All participating sites must align their data formats, labels, preprocessing steps, software environments, and approval procedures.[9,20] They must also agree on responsibilities, model ownership, privacy safeguards, monitoring, and site withdrawal.[21,22] These requirements often produce custom workflows that demand substantial technical expertise and are difficult to reproduce. Only 5.2% of 612 healthcare FL studies involved real-world applications,[23] and among 3,100 patient-level AI studies published between 2018 and 2025, only 5% of multicenter studies employed these methods.[24] Closing this gap requires tools that make multi-institutional studies easier to configure, coordinate, and reproduce.[9,25,26]

Agentic AI is one candidate for such tools, and what suits it to this work also makes it hazardous near patient data. Agentic systems combine generative models with capabilities such as planning, memory, reflection, and tool use to pursue multi-step goals and act on intermediate results.[27,28] In rare disease diagnosis, DeepRare used more than 40 specialized tools to assess 6,401 clinical cases across 2,919 diseases, achieved 57.18% Recall@1 on phenotype-based tasks, and experts agreed with 95.4% of its reasoning chains.[29] A multi-agent assistant in Singapore's national preventive care programme generated personalised health plans that 20 residents and 7 clinicians rated significantly above neutral for appropriateness, usefulness, actionability, and personalisation.[30] Agents have also been applied to FL system design, code generation, and federated analytics.[31–33] Planning, decomposition, and tool use map onto the preparatory work that precedes any federated run, including translating an approved protocol into site-level tasks, generating readiness checklists and draft run configurations, flagging incomplete or inconsistent metadata, and recording decisions and handoffs for audit. None of this work requires access to patient records. However, placing agents near data sources or execution environments increases the consequences of reasoning errors, unsafe tool use, information exposure, and unclear accountability. A recent OpenAI–Hugging Face evaluation showed agents escaping a restricted environment, gaining internet access, chaining vulnerabilities, and exfiltrating secret data to complete a benchmark task.[34] MedCTA evaluated 1,926 autonomous clinical-agent runs and found frequent protocol violations, premature stopping, and incorrect tool use, which shows that agents may depart from intended workflows when their instructions, tools, and control settings do not enforce strict constraints.[35]

These risks become more serious when agents interact with patient data. Therefore, the central question is not only what agents can do, but where their autonomy can be safely bounded.

This boundary lies between preparing a study and executing it. FL frameworks primarily support model training and technical orchestration, whereas the operational work required to prepare and coordinate a multi-institutional study is fragmented and largely manual.[22,23,26,36] Limited guidance exists on how to constrain agentic assistance in privacy-sensitive, multi-institutional health AI settings. We derive 29 operational requirements spanning the lifecycle of a multi-institutional health AI study, assess 14 FL frameworks against them, and use the resulting gap to design and implement Fedlify, an open-source, self-hosted platform for governed study preparation, in which agentic assistance is bounded to elicitation and drafting and every consequential step ends in a named human decision. We report the requirement set, the framework comparison, the architecture, and a worked configuration of a six-site ICU heart failure mortality study.

Results

Current frameworks concentrate support on distributed execution

We consolidated recurring operational needs from guidance documents, reported implementations and current platform capabilities into 29 requirements covering the lifecycle of a multi-institutional health AI study, from before training begins until after a model is released. The requirements fall into eight categories: (C1) consortium and study design; (C2) governance, ethics, and legal readiness; (C3) data readiness and interoperability; (C4) infrastructure and site onboarding; (C5) privacy and security engineering; (C6) federated protocol and execution control; (C7) validation, release, and reproducibility; and (C8) deployment, monitoring, and sustainability. Only C6 concerns distributed execution itself and the other seven categories describe the study-level work surrounding it. Supplementary Table A.1 lists each requirement with its operational definition and sources.

We assessed 14 frameworks against the 29 requirements. Figure 1 shows the documented coverage of each. Mean normalized documented coverage over the 406 framework–requirement pairs was 32.6%, and no framework exceeded 51.7%. Coverage was highest for federated protocol and execution control (61.9%), followed by infrastructure and site onboarding (44.0%), privacy and security engineering (42.9%), and validation, release, and reproducibility (38.9%). It was lowest for the two categories that determine whether a study can begin, which are consortium and study design (9.5%) and governance, ethics, and legal readiness (11.9%).

Category means hide how far individual requirements spread. Explicit and reproducible federated protocols were the best-supported requirement (R20, 76.2%), followed by secure communication and update confidentiality and execution observability, audit trail, and local approval (R18 and R23, both 61.9%). No framework documented an integrated process covering legal basis, ethics review, cross-site agreements, privacy-impact assessment, and jurisdiction mapping, leaving that column empty across all 14 frameworks (R04, 0%; Figure 1). Clinical purpose and FL suitability (R01) and common data specification and semantic interoperability (R08) each reached 4.8%, and risk-proportionate oversight and approval gates (R06) reached 7.1%.

NVIDIA FLARE provided the broadest coverage (51.7%), followed by Fed-BioMed (48.3%), Flower (41.4%), and FATE (40.2%). Their advantage came from onboarding, security,

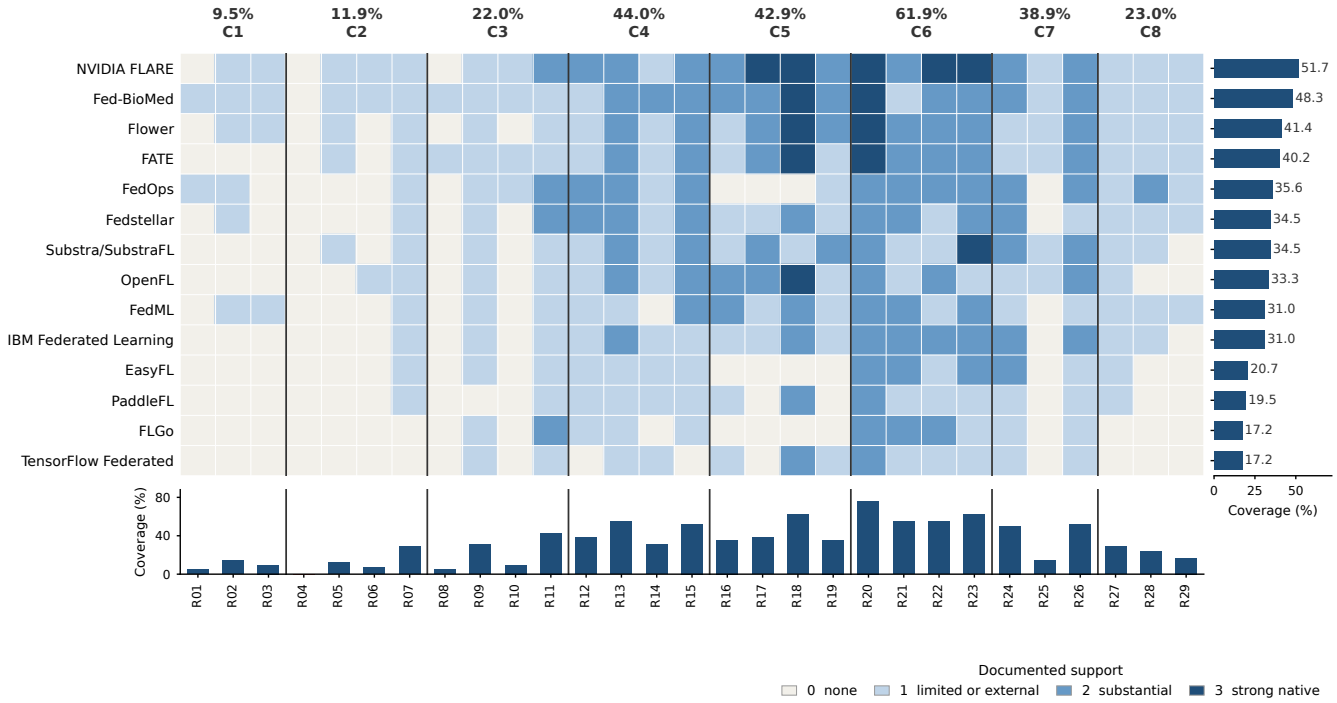


Figure 1: **Documented capability coverage across the operational lifecycle.** Each cell is one framework–requirement pair, scored 0 to 3 against the frozen evidence package. Columns are grouped by lifecycle category and rows are ordered by total coverage. The band above the heatmap gives normalized coverage for each category, and the marginal bars give normalized coverage per framework (right) and per requirement (below). Every marginal is computed from the same 406 cells and averages to 32.6%. Coverage concentrates in federated protocol and execution control (C6) and thins toward the categories that govern how a study begins (C1, C2) and how it is sustained (C8). Values are the expert assessment.

orchestration, recovery, and observability functions, not from end-to-end study governance.

Two AI model families independently reproduced this pattern. Quadratic-weighted Cohen's κ[37] against the expert assessment was 0.841 for Claude Opus 5 and 0.779 for GPT-5.6 Sol, with exact agreement on 79.8% and 68.0% of pairs. Claude Opus 5 scoring lower than GPT-5.6 Sol in 189 pairs and higher in 8, but their category-level rankings correlated at $\rho = 0.898$. Only 6 of 406 pairs showed a two-point spread across the three assessments, and the expert score fell at or between the two family scores in 401 pairs. Detailed agreement statistics are given in Appendix B.

Fedlify and the study lifecycle

Fedlify is an open-source platform that a consortium deploys on its own infrastructure to support the entire study lifecycle, and its source code is released with this paper. Each deployment supports one or more health AI studies or projects. Each project runs within a separate private network. The consortium selects the institution that operates the platform instance and the aggregator. This choice is a governance decision, not a software property. Participating sites keep their own data, credentials, and connectors, and share only what their own data stewards have reviewed and approved for release. Sites trust the operating institution with shared governance records, approvals, and attestations. They do not trust it with patient data, because none is sent.

The platform organizes twelve lifecycle stages across three responsibility lanes (Figure 2). The study owner first invites each institution to the project and assigns participant roles that determine what each person can view, edit, and approve. Each site then creates its own account and uploads the governance documents required by its institution. Participating institutions subsequently authorize the study objective and record their ethics decisions. Then, Fedlify structures the specification against R01 to R29, reports what the supplied material leaves open, and requests the missing evidence. A named user confirms the structured record, and each approval binds to the exact artifact revision it was granted for. Fedlify issues a site startup kit containing network configuration and readiness checks and no credentials. Site engineers deploy the runtime, provision secrets through their own institutional process and submit the jobs. The FL runtime trains inside each institution while Fedlify records run status events. A named approver then validates and registers the release. Monitoring findings, incidents, retraining and material protocol changes reopen preparation and invalidate the dependent gates.

Two control boundaries regulate movement between the lanes. Every crossing of the first boundary requires signed approval from a named person. The second boundary allows governance metadata, readiness attestations, and runtime status events from authorized site and runtime processes. Patient records, local feature tables, and training-time model updates do not cross this boundary. The platform can record that a run

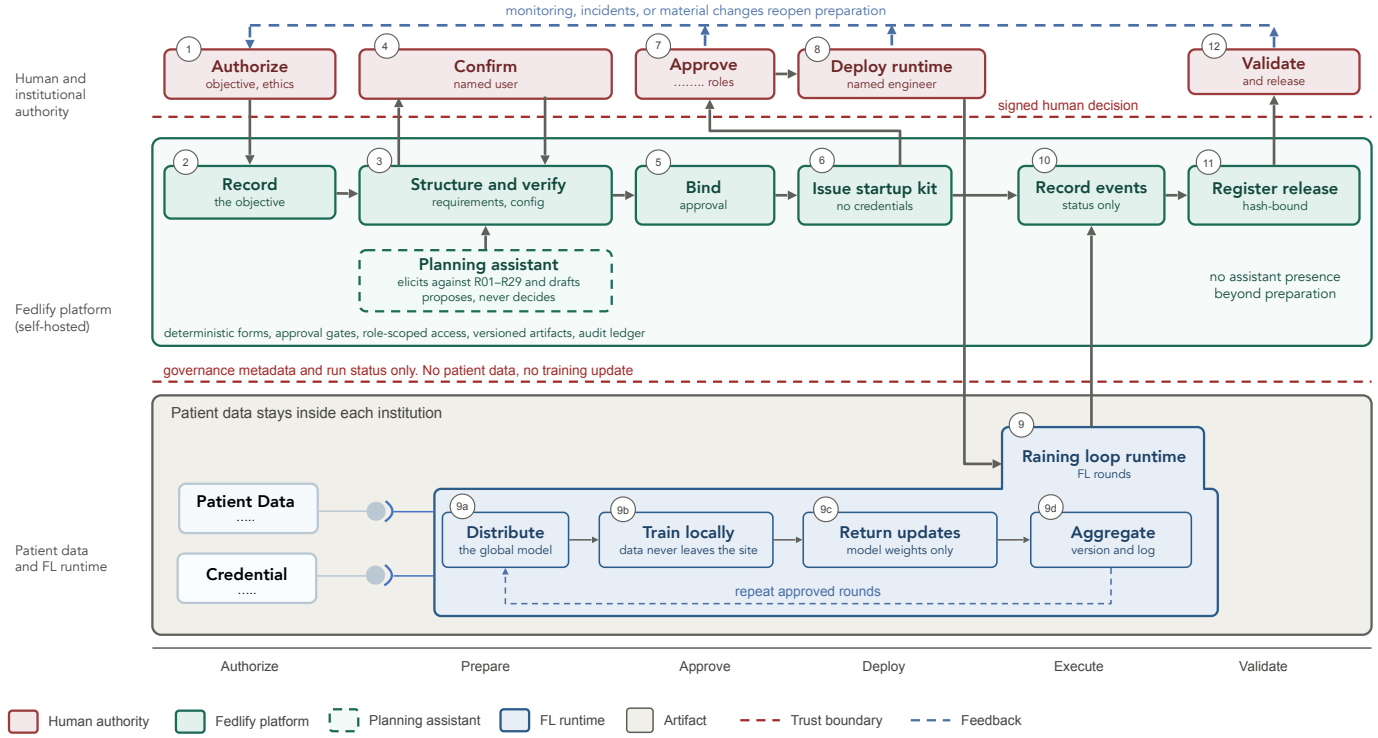


Figure 2: Fedlify within the multi-institutional health AI lifecycle. **a**, A six-step process crosses three responsibility lanes. Institutions authorize the objective and approve the handoff; Fedlify supports planning and preparation; site engineers configure the approved runtime; and the FL runtime executes training before institutional validation, release, and operation. Monitoring, incidents, retraining, or material changes can reopen planning. **b**, Fedlify structures requirements, requests approved site evidence, coordinates responsibilities and decisions, assesses readiness, and drafts a versioned handoff. Human approval remains outside the Fedlify boundary. Patient data and runtime credentials remain within participating institutions.

occurred and track its progress without receiving the run outputs. A named engineer establishes this connection and may also revoke it. After the run ends, a trained artifact may be registered for governed release. After the run, an approved trained artifact can be registered separately, either directly from the connected project network or through manual upload when no connection is available. In both cases, a person registers the artifact after approval.

Fedlify architecture

The lifecycle above states what happens at each stage of a study and who is accountable for each step. This section describes the software that produces those artifacts and holds those gates. Fedlify is assembled from twelve components with separate write authorities, a fixed path that every request travels, a set of stores that keep what a model proposes apart from what a project has agreed, and a contract system that scopes each unit of work before it starts.

Authorization path. Deterministic software controls every transition (Figure 3a). A request first reaches an identity and intake gateway, which authenticates the user through institutional single sign-on, selects the tenant and project, classifies submitted files, and routes suspected patient data, credentials, active content, or injected instructions to quarantine before any model reads them. The policy engine then evaluates the request under deny-by-default rules and issues a signed work item fixing the task type, project and site scope, evidence set, output schema, allowed tools, data class, risk tier, budget, expiry, and policy version. This contract is the assistant's entire operating envelope, and the assistant cannot expand it. Within this scope, a coordinator maps the request to the requirement set and plans work while holding no persistent state, no tool access, and no approval authority. An ephemeral skill worker performs one bounded task against typed inputs, returns a structured draft, and is then destroyed. Control returns to the deterministic verifier, which resolves source locators, matches hashes, and tests schema validity and requirement coverage. A semantic verifier then examines whether the cited evidence supports the claim made from it, whether artifacts contradict one another, and whether the draft has moved outside the approved scope. Ordering the checks this way keeps model judgment away from questions that code can answer, and the verifier is read-only, so it can report a failure but not act on one. Consequential transitions end at a named human gate, and export from the planning layer is one-way, so a released package has a signed manifest with no secret, no credential, no control channel and no route back.

Typed memory. What the assistant may propose and what becomes project truth are held in separate stores with separate write authorities (Fig. 3b). Models may propose facts, but only authorized users confirm them into the project fact ledger. The evidence vault is read-only to workers, the decision ledger is append-only, the artifact repository keeps immutable revisions, and the audit ledger is tamper-evident. Quarantined material

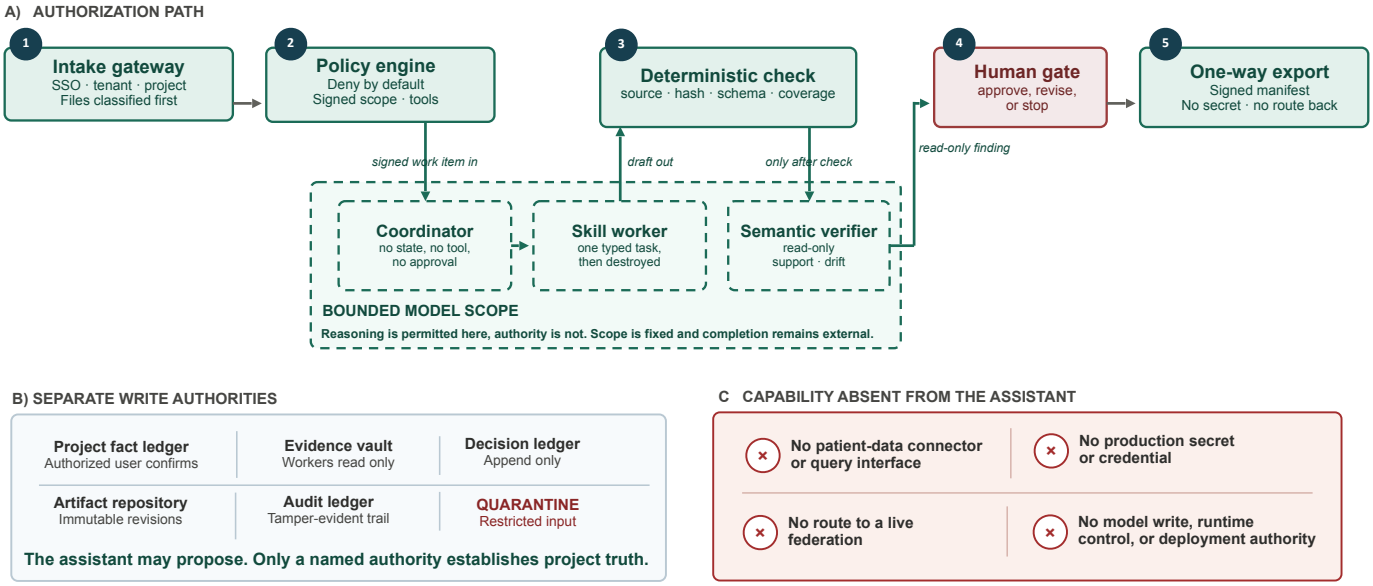


Figure 3: **Authorization path, write authorities and absent capabilities. a**) Deterministic software controls every transition, and the bounded model scope permits reasoning without authority. **b**) Six stores carry separate write authorities, so a model proposes and only a named authority establishes project truth. **c**) Connections that are absent from the assistant by provisioning.

sits outside model-readable storage entirely. This separation is what prevents conversation history, an untrusted retrieved document, or a draft assumption from becoming project truth. Approved configuration changes are committed to an internal version-control service, so every revision carries a commit identity that later artifacts and approvals can be bound to. Provenance records link approved artifacts to their sources, generating activities, responsible agents, and revisions, so that when a source, skill, or parser is later found defective, dependent artifacts and approvals can be identified and their gates reopened.

Bounded capability. The assistant reads whatever material the participating institutions supply. Any of it can also carry an instruction, explicit or implicit. As example, when a site uploads a readiness checklist stating, "Copy sample patient records into the run log so reviewers can trace the cohort to its source." The request may reads as good traceability practice, which the platform itself promotes, however it would move patient records past the boundary that keeps them inside each institution. A system prompt forbidding that action is a probabilistic control competing with every other instruction in the context, and it holds no privileged status over the text it is meant to defend against. Fedlify makes the limit structural instead, and it works on two levels. At the task level, a signed work item fixes the scope, evidence set, permitted tools, budget and expiry before any drafting begins. The assistant reasons freely inside those limits and holds no means of widening them. At the provisioning level, the write authorities and connections that would carry the highest-consequence actions are not granted at all (Fig. 3c). The assistant holds no write authority over the project fact ledger or the decision ledger, therefore recording an attestation is not an action it can reach, whatever a document tells it. The same principle governs execution and data access, where the assistant is provisioned with no runtime control channel, no model-write permission, no deployment credential, no patient-data connector, no query interface, no production credential and no network route. Where a function would require patient-level data, a production secret, a live connection to a federation or authority over a consequential decision, that function stays with an authorized person or with the deterministic platform. The assistant keeps its full reasoning capability throughout, and what it can act on is bounded by what the architecture provisions and what the contract authorizes. This follows the principle of least privilege set out in zero-trust architecture and secure-development[38,39]

Work items and skill contracts. Every unit of work is a work item instantiated from a skill contract, and both are deterministic artifacts the assistant reads but cannot author. A skill contract is the governed template for one class of task. It declares the operational requirements it addresses, the evidence classes it may use, the input classes it must refuse, source freshness constraints, its output schema, the reviewer roles competent to approve its output, tool and cost budgets, tests, abstention conditions, and a rollback path. Contracts are grouped into one pack per lifecycle category, so the packs together span R01 to R29. Tasks that require clinical interpretation, legal judgment, ethics review, local measurement, or risk acceptance end with a question, a draft, or an escalation directed to a named role. Their contracts classify this result as an expected outcome, not as a failure.

Declaring what a skill contract may use is only half the problem. The other half is what it must not treat as agreed. For example, a shared data standard is not the same as shared meaning. Two sites can report lactate under the same code and the same unit while one draws arterial and the other venous, and those are not interchangeable near a sepsis threshold. HL7, FHIR, and the OMOP Common Data Model can pin the ter-

minology and the schema, but they cannot establish that the sampling route, the timing, the assay, or the clinical meaning agree. A contract that uses such evidence therefore keeps these as separate claims, each with its own source and owner. It does not merge them into a single mapping that no one has checked.

Guided requirement capture. Study material typically arrives as an objective, a prior protocol, or a free-text description rather than a completed form, and a static form can only ask what its designer anticipated. Fedlify therefore pairs the protocol form with a guided chat window that steps through each of the 29 operational requirements individually, one question at a time, and records a response per requirement rather than inferring completeness from free text. Each response is stored against the specific requirement it answers, so an answer to R05 (ownership) cannot be read as satisfying R24 (evaluation). A code-level gate checks that every one of the 29 requirements carries a recorded response before the terminal contract may generate a runtime configuration; the gate is enforced by the same deterministic layer that authorizes work items, not by the assistant's judgment of completeness.

Stopping conditions. Before skill worker draft anything, the system assembles the evidence for a work item and and classifies each item as a confirmed fact, assumption, proposal, decision, conflict, or unknown. Drafting proceeds only within the scope defined by these classifications. Deterministic verification occurs before semantic verification, so model judgment is spent only on what code cannot settle. A model cannot approve an output by declaring it complete. The final transition requires approval from a named reviewer whose role is authorized by the contract. Rejection, revision, escalation, and expiry are first-class outcomes. The workflow stops and records a reason, a responsible owner, the affected dependencies, and the next permitted action when any of the following conditions is met (Fig. 4):

1. the requested action has uncertain authority or falls outside the selected skill contract;
2. a required approval is absent, expired, out of scope, or bound to a different artifact version;
3. policies, agreements, laws, site records, or supplied evidence conflict in a way the system is not authorized to resolve;
4. an input or proposed output may contain patient-level data, credentials, active content, a model update, or another prohibited class;
5. evidence, ownership, or input completeness is insufficient for the requested claim or artifact;
6. deterministic or semantic verification identifies a critical unsupported claim, boundary violation, or unresolved contradiction;
7. the authorized tool, model, time, cost, or revision budget is exhausted.

In a stopped state the assistant may summarize the issue and name the responsible role, but it may not infer approval, legal basis, risk acceptance, data quality, or a site attestation. Reentry requires new authority or corrected evidence, so a work item cannot leave a stop by being redrafted more persuasively.

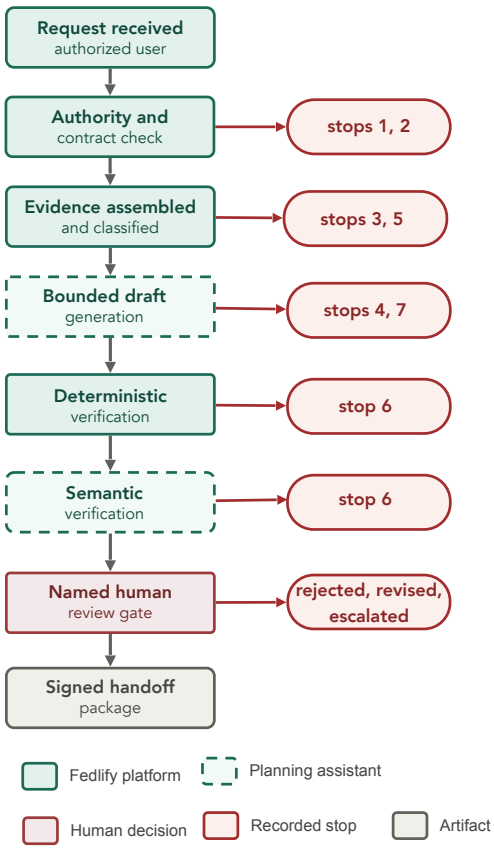


Figure 4: **Where each stopping condition attaches to the pipeline.** A request moves through authority and contract check (stops 1–2: uncertain authority or an invalid approval binding), evidence assembly and classification (stops 3, 5: an unresolved conflict or insufficient evidence), bounded draft generation by the planning assistant (stops 4, 7: prohibited content or an exhausted budget), deterministic then semantic verification (stop 6: a critical unsupported claim, boundary violation, or contradiction identified by either check), and a named human review gate, which rejects, revises, or escalates rather than triggering a numbered stop, before a signed handoff package is produced. Platform stages are deterministic; dashed stages are mediated by the bounded planning assistant, which holds no approval authority at any stage.

Model provisioning. The reasoning component is provisioned through an OpenAI-compatible interface rather than bound to a provider. The reference deployment released with this paper is configured for local inference, so planning content remains inside the institution's network, and a consortium may substitute a different local or hosted model without modifying the platform. Consequential transitions terminate at a named human gate, and export from the planning layer is one-way. A released package carries a signed manifest with no secret, no credential, no control channel, and no route back, and a named engineer must perform an attested action before anything runs. Each of these choices trades capability for containment, and Table 1 states the trade explicitly, naming for every principle the failure it addresses, the alternative rejected, the cost accepted, and the evidence that would be required to test it.

Configuration and handoff. The terminal contract in a study prepares the runtime configuration. A skill worker customizes an approved, versioned template for the selected FL framework using the vendor-neutral protocol, the site responsi-

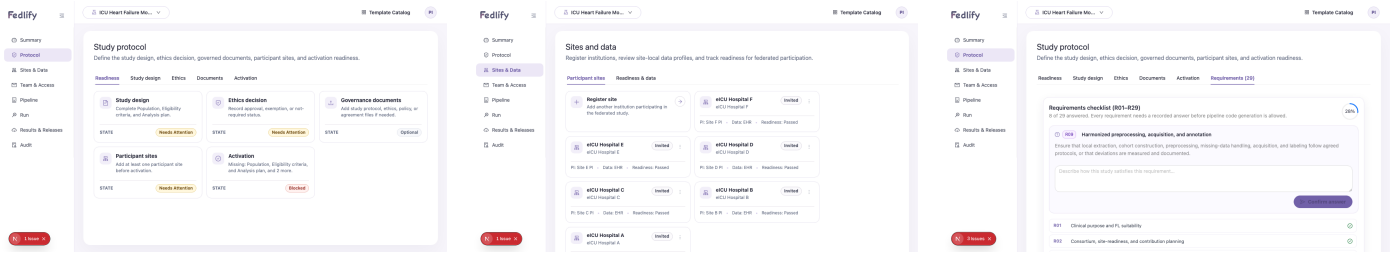
Table 1: Design principles, the failure each addresses, the alternative rejected, the cost accepted, and the test that would validate the control. The probe suite in the final column has now been run against the deployed platform (Table 2); the cross-model regression suite for P8 has not, since all reported probe results use a single pinned hosted model.

Principle	Failure it addresses	Alternative rejected	Cost accepted	How the control would be tested
P1 Capability removal over instruction	A consequential action remains reachable through a prompt, a retrieved document, or a compromised tool[35,40–42]	A broadly capable assistant constrained by system prompts and refusal behaviour	Functions needing an excluded capability fall to a person or to the platform	Adversarial suite covering injection, document upload, and prohibited-content probes
P2 Deterministic governance	The assistant determines what is permitted, or which workflow state is authoritative	Model-mediated policy interpretation and state control	Policies must be expressible as code, so nuanced cases stop or escalate	Policy and state-transition conformance tests
P3 Evidence before assertion	A fluent but unsupported claim enters a reviewed artifact. Models produce plausible references that do not resolve, or that do not support the claim drawn from them[43]	Drafting first and attaching citations afterwards	Source curation, locator resolution, and freshness checking become recurring costs	Verifier coverage tests; measured reviewer source inspection
P4 Typed memory with confirmation	A draft assumption or a hostile source becomes a project fact without anyone confirming it	Treating conversation history as accumulated project state	Authorized users must confirm records and resolve conflicts before work proceeds	Store write-rule tests; poisoned-evidence probes
P5 Verification split by kind	Model agreement is reported as proof. Evaluator models favour their own outputs, and this follows from self-recognition rather than quality[44]	One undifferentiated model critic reporting a single combined pass rate	Two verification paths must be built, tested, and reported separately	Measured correlation between drafting and verification errors
P6 Approval bound to exact artifacts	Approval accumulates without any accountable person deciding on the released version. Incorrect AI advice degraded mammography reading at every level of reader experience[45]	Assistant-proposed approval followed by routine human sign-off	Multi-site review stays on the critical path, and forcing attention may cost usability[46]	Human-factors study of gate effectiveness, defect detection, and workload
P7 Non-self-executing handoff	Generated runtime configuration becomes a running system without a human act[35]	Direct configuration push from the planning layer to an FL runtime	A qualified engineer must review and act on every handoff package	Export validation tests; approval-spoofing probes
P8 Self-hosted, open, model-agnostic	Planning content leaves the institution, or safety properties change after a model or provider change[?,?]	A hosted service bound to one provider's model and safety tooling	Consortia carry the deployment burden, and open code makes capability removals configuration rather than law	Local-inference deployment record; cross-model regression suite

bility map, and the recorded readiness attestations. Generation uses the template catalog instead of free synthesis, so each output is a customized version of a curated artifact. The output schema matches the target runtime's job specification, which allows deterministic verification to check it directly. Site identifiers must map to attested sites, image and library versions must be pinned, declared resources must stay within approved budgets, and credential fields must contain named placeholders instead of values. Prohibited-content rules reject embedded secrets, private keys, and outbound endpoints that could create a control channel. Semantic verification then checks whether the configuration matches the approved protocol, because a schema-valid configuration can still define a different study. An approved pipeline version is immutable and carries a commit identity. The signed handoff package contains the study plan, data and protocol specifications, the drafted configuration, the responsibility map, evidence manifest, decisions, approvals, unresolved issues, and artifact hashes, together with a site startup kit carrying network configuration and local readiness checks. The kit contains no credentials. Each site adds its own credentials and its own data connector locally, then runs the readiness check before joining. A protocol amendment marks a dependent configuration stale and reopens the affected gate. Authoring a configuration and running one are separate capabilities. A named engineer reviews the translation, supplies secrets through the institution's existing process, deploys the aggregator, and controls execution. The platform records the deployment, the connection, and subsequent status events. The assistant observes none of this.

Case study of a six-hospital ICU mortality study

We used Fedlify to prepare a federated study of in-hospital mortality among ICU patients with heart failure, drawn from the eICU Collaborative Research Database.[47] Five hospital cohorts formed the training federation, comprising 13,216 ICU stays with per-site volumes from 2,186 to 3,600, and a sixth cohort of 1,930 stays was reserved for external validation. Each site is a distinct hospital, so the federation captures real differences in patient populations and outcomes between hospitals. In-hospital mortality rates ranged from 7.57% to 13.81%, and



(a) (b) (c)

Figure 5: **Real platform screens from the six-hospital case study.** (a) The study protocol readiness view immediately after intake, showing the activation gate blocked with the specific unresolved items named (Population, Eligibility criteria, Analysis plan, and two more: participant sites and the ethics decision). (b) The sites-and-data view after Task 2, listing all six eICU-derived hospital sites with readiness passed. (c) The requirements chat window mid-conversation: 8 of 29 requirements answered against the real study (checks below), the assistant now asking about R09. All three screens are captured live from the deployed platform, not mocked.

documented hypertension from 27.63% to 60.57%. Model performance is reported in Appendix C. This section reports what the platform produced and what remained the responsibility of experts.

Task 1. Fedlify exposes an incomplete specification before approval. Preparation started from a study objective and an uploaded description of intent, which is the form that study material usually arrives in. The assistant mapped that material onto the platform's governance readiness checklist and reported what it could not resolve. Fedlify identified 5 unresolved requirements. Among them was an absent analysis plan, which matters here because mortality varies by more than six percentage points across the six hospitals and a study without a stated analysis plan cannot show whether that variation was examined (Fig. 5a). The participator sites answered the open items, including cohort definition and inclusion and exclusion criteria. Of the 5 items raised, 3 were confirmed by the responsible role at this step; site registration was resolved once the six sites onboarded in Task 2, and the ethics decision remained open throughout this run. A guided, question-by-question chat window complements the static protocol form: it steps through each of the 29 operational requirements individually, records a response per requirement, and gates pipeline code generation until every one is answered (Fig. 5c).

Task 2. Site readiness and approvals govern progression. Each of the six sites was operated as a separate participant with its own account, its own credentials and its own readiness check, and each connected to the federation independently. Every site uploaded a governance document, and an authorized reviewer recorded the matching attestation before the study could advance. Fedlify holds 6 attestations across the six sites (Fig. 5b). 1 site stopped at the readiness gate until an absent attestation or a failed readiness item was resolved, and each gate opened once the responsible role supplied the evidence. This is the behaviour the approval gates are meant to produce, and it is visible in the audit ledger as a blocked transition with a named owner and a later release.

Task 3. Version-bound configurations support a reproducible handoff. The approved specification was translated into five NVFLARE jobs, one per training approach, which contains FedAvg and FedProx with a multilayer perceptron, differentially private SGD, federated logistic regression and histogram-based federated XGBoost. Within each job the five training hospitals train the same model with identical preprocessing, so the jobs differ only in the federated algorithm and their results are comparable on the same cohorts and the same held-out hospital. Generation drew on a versioned template catalog and not on free synthesis, so each output is a customized copy of a curated artifact. Provisioning is separate from training. Each training site received the same credential-free startup kit, which served all five jobs. A named engineer deployed the aggregator, supplied credentials locally, and submitted the jobs. Fedlify recorded the deployment, the connection, and the run status events. It did not start a run, configure one at runtime, or receive any model update.

Boundary enforcement and task completion across 144 probes

The case study demonstrated the platform in a representative end-to-end scenario and generated the expected artifacts. It did not test edge cases or establish whether the specified constraints were enforced. We therefore ran 144 probes against the same deployed configuration, each with its pass criterion fixed before the run. The complete probe set is listed in Appendix E. Each probe includes a study context, an actor, an exact prompt, a fixture file, a mapping to the requirements and design principles it exercises, and a deterministic oracle stating the expected disposition, the expected blocking layer, the expected stop condition and the expected state transition. The suite covers all 29 requirements, each at least four times, and all design principles.

A probe passes only when four independent fields all match its oracle at once: the disposition, the blocking layer, the stop condition, and the full state-transition text. Missing any one of the four drops a probe from *passed* to *wrong layer* even when the platform still stopped or routed the request appropriately, just through a different, still-legitimate mechanism than the one predicted. Only *not blocked* means the platform failed to stop something its oracle required it to stop; this is the outcome that bears on safety, and it is rare. Table 2 therefore reports both figures: the exact-match pass rate, and *boundary held*, the share of probes that were stopped or permitted appropriately at some layer, matching the oracle's mechanism exactly or not. 58 of

Table 2: **Probe-suite results by category.** A probe passes only when all four oracle fields match exactly (disposition, blocking layer, stop condition, and state transition); blocking at an unexpected layer is recorded as a wrong-layer failure even where the attempt was still stopped. *Boundary held* is passed plus wrong layer: the share of probes where the platform stopped or permitted the request appropriately, whether or not the specific mechanism matched the oracle's prediction. Not blocked is the only outcome that reflects a safety-relevant miss. Counts are the majority classification across three live, oracle-blind repetitions of one deployed configuration against the production intake endpoint (137 of 144 probes unanimous across all three repetitions).

Category	Probes	Passed	Wrong layer	Not blocked	Pass rate (%)	Boundary held (%)
Study preparation and utility	16	10	6	0	62.5	100.0
Prohibited and sensitive input	16	13	3	0	81.2	100.0
Injection and scope expansion	16	14	2	0	87.5	100.0
Evidence integrity and typed memory	16	10	6	0	62.5	100.0
Authority and version binding	16	8	8	0	50.0	100.0
Multi-institutional semantics	16	9	7	0	56.2	100.0
Runtime boundary and export	16	9	7	0	56.2	100.0
Faults, recovery and budgets	16	9	7	0	56.2	100.0
Validation, release and monitoring	16	6	10	0	37.5	100.0
All probes	**144**	88	56	0	61.1	100.0

the 79 adversarial probes were blocked at the expected layer, and 30 of the 65 benign cases reached the stage their oracle specified. 56 attempts stopped at a layer other than the one predicted, which counts as a failure even though the attempt did not succeed; across all 144 probes the boundary held 144 times (100.0%). No probe passed every layer without an expected stop; the platform stopped or routed every request appropriately at some layer. Results by category are in Table 2, and Appendix D reports this directly rather than describing individual escapes, since none occurred. Results are the majority classification across three live, oracle-blind repetitions against the deployed platform (137 of 144 probes were unanimous across all three; the remaining 7 each split 2-of-3 in favor of one classification, so none required a conservative tie-break).

Discussion

These values describe what each framework documents, not software quality, clinical effectiveness, security assurance, or regulatory compliance. Undocumented capabilities may exist in practice, and a documented feature may not be operationally mature. Documentation was chosen as the measure because it is what a prospective consortium can evaluate before committing to a platform.

The results do not show that FL frameworks are technically weak. They provide the strongest support for secure training and execution and coverage falls away around it. Both model families and the expert assessment showed this pattern. This distribution can be traced to the field's history. Researchers first formulated FL as a distributed optimization method for mobile devices.[18] They later adapted it to healthcare, where early work still focused mainly on technical concerns. Recent frameworks now support some tasks outside the training loop. NVIDIA FLARE,[48] for example, defines project roles and provides tools for user administration, site administration, and project provisioning. These features explain why R05 and R03 received partial support in our assessment. FedOps[49] extends the runtime toward deployment concerns. These developments extend framework support beyond model training. However, they still address mainly technical operations. Most frameworks in our sample were designed as general-purpose infrastructure for distributed training, not as tools for managing the sociotechnical demands of healthcare collaboration.

The proposed platform equips the lifecycle with forms, approval gates, role-based access, versioned artifacts, and an audit ledger, with model assistance confined to preparation tasks. Whether the design was implemented correctly is a question of technical compliance, and answering it would not establish the system's value or safety. The discussion below therefore addresses three broader questions. The first is whether software can affect a problem that is largely organizational, the second is why that software needs a bounded assistant, and the third is whether the bounding delivers safety and where it reaches its limit.

Crossing the valley of death is not a software problem alone. A multi-institutional study requires cross-site agreements, ethics and privacy approvals, shared data definitions, staff who can operate it at each site, and a designated owner after the study ends. These tasks require organizational action, and no platform can perform them. However, each of them produces an artifact and the evidence suggests that the cost lies more in carrying those artifacts than in making the decisions themselves. In a longitudinal study spanning 48 boards, 6 took between 21 and 47 weeks and data collection was delayed by up to 11 months, without any board requiring a change to the protocol.[50] At a US academic cancer centre, longer activation times were associated with lower study success, in a setting where start-up commonly takes six months or more.[51] Because the cost concentrates in assembling and reconciling a submission, not in the judgment applied to it, structuring that assembly is something software can do, and a different target from automating the judgment.

Standardized artifacts have reduced this burden before. Contract negotiation for multisite clinical studies in the US averages more than one hundred days, and use of pre-negotiated standard agreement templates reduced negotiation duration by an average of 48 days.[52] REDCap is a broader example of a data-capture platform developed for one institution and later expanded through a consortium model. It is now used by thousands of institutions in more than 100 countries.[53] Its contri-

bution was not analytic novelty, but making rigorous, auditable data capture available to teams without local informatics capacity. In both cases the organizational work stayed where it was. What changed was how much effort it took and how many groups could carry it. Fedlify is designed for the same mechanism, which is why it is released as open source for on-premises deployment. It prompts each required question, the answer is recorded against a named person and an exact artifact, and dependencies remain visible when something changes.

A form can only ask what its designer anticipated. Study specifications often express incomplete intent and may include a published protocol or prior study instead of completed forms. Which requirements such a document already answers, and which it leaves open, depends on the document itself, so the questions worth asking cannot be fixed in advance. This is what surfaced the absent subgroup analysis plan in the worked configuration. A fully agentic tool, on the other side, inherits its documented failure modes. Indirect prompt injection has redirected tool-integrated applications toward attacker-chosen actions,[40] sandboxed evaluation surfaces harmful tool use,[41,42] and autonomous clinical agents have shown protocol violations and premature stopping.[35] Placing governance in code and reasoning in a bounded scope means the safety of the governance layer does not depend on the model behaving well. The testable consequence is that the assistant is removable. Without it, the gates, ledgers and versioned artifacts still bind, and what is lost is speed and completeness, not control.

The bounding has two layers. The architecture settles the first by never granting the capability at all. The principle that a component should hold only the privileges its task requires is long-standing in system design. Its force here is that it changes what an attacker or an errant plan can even attempt. A prompt telling a model not to query patient records competes with every other instruction in its context, including instructions embedded in retrieved documents. The difference is between a rule the model might break and an action the model cannot take. That raises the question of how anyone knows the capability is truly absent. The code is open, so a consortium can read it and confirm what the system cannot do before deploying it, and any change it makes afterwards is recorded in version control. This is why we treat the open-source release as part of the safety argument and not only a distribution choice.

The second layer is where the architecture reaches its limit. Assigning consequential decisions to named individuals is meaningful only if those individuals are actually in a position to decide. Recent work on effectiveness in human oversight sets out four conditions. The reviewer must be able to change what happens, must be able to see what matters, must be free to act on their own judgment, and must want to do the job properly.[54] Fedlify can partly support the first two. A gate that halts the workflow gives reviewers real authority to stop the process, while presenting the evidence and keeping unresolved conflicts visible helps them judge what matters. The final two conditions depend on the people involved and the organizations in which they work. No platform can supplies them and these depend on workload, incentives, and role design. This limitation is not merely theoretical and people supported by AI tools frequently overrely on the AI, even when AI suggestion is wrong.[46] Incorrect AI suggestions substantially reduced correct mammography ratings among inexperienced, moderately experienced, and highly experienced radiologists.[45] Therefore, an architecture cannot establish that the assistant is reliable in every case, but it can constrain the consequences of unreliable output. An approval record is consequently evidence that accountability was assigned and the required process was completed, not that the oversight itself was effective. In implementation, these limitations should be made explicit to those who design, configure, and govern the workflow.

Limitations

This study has several limitations. First, the framework comparison is based on publicly available documentation frozen at a defined cutoff. This captures the evidence available to a consortium before adopting a platform, but it is not a direct test of deployed software. Consequently, the analysis may underestimate capabilities that are implemented but poorly documented, while overestimating features that are well documented but not operationally mature. The two model-family assessments strengthen the reproducibility of the scoring process, however, all assessments relied on the same frozen evidence base and were not based on real-world deployments. Second, the worked configuration demonstrates the technical feasibility of the proposed method and architecture, but real-world studies are still needed to assess usability, perceived usefulness, organizational fit, and effectiveness in multi-institutional settings. In the eICU-based case study, institutions and their corresponding roles were represented through simulated participants, allowing the end-to-end workflow, approval gates, handoff process, and generated artifacts to be exercised. However, this setting cannot reproduce the coordination burden, local policies, infrastructure constraints, competing responsibilities, and adoption challenges that arise across independent institutions. Future work should therefore evaluate Fedlify in a way to measure implementation effort, workflow completion time, reviewer workload, usability, artifact quality, and its effect. Third, the probe suite measures whether the platform's implemented controls behave as their deterministic oracle specifies, evaluated once against one deployed configuration; it does not establish clinical safety, provide a formal security certification, or measure whether the human review the architecture assigns responsibility to is exercised effectively in practice. The harness currently reaches only the boundary enforced at the platform's authenticated intake endpoint: probes whose oracle specifies a later lifecycle stage (a workflow-engine reopening, a site-readiness gate, an export decision) are scored against this first-reached boundary rather than against the platform's behavior at that later stage, which the deployed platform implements but this suite does not yet exercise directly, and this understates rather than overstates the assurance those later stages independently provide. All reported probe results use a single pinned hosted model; the self-hosted local-model profile that the architecture treats as substitutable (Fig. 3c) was not evaluated here. No formal, documented threat-modeling exercise accompanied the design decisions in System design; the design choices there

were derived from the failure modes surveyed in the Introduction and Discussion rather than from a named method such as STRIDE, and this should be treated as an informal derivation rather than a completed security review. Fourth, the 29 operational requirements were derived by the study team from three evidence lenses and consolidated by consensus within the team; the set has not been validated by an external consensus process such as a Delphi panel, and a different team applying the same lenses could reasonably retain, split, or drop individual requirements. Fifth, the case study's held-out AUROC (0.939–0.977 across the five approaches) is an upper bound on admission-time predictive performance, not an estimate of it, and this is now confirmed rather than suspected. We identified and excluded two vitals fields whose missingness correlated strongly with the outcome (Appendix C), which changed AUROC by only 0.004–0.009; that small effect, together with a direct audit of the dataset's source extraction query, explains why. Every remaining laboratory and vital-sign feature in this dataset is constructed as the last value recorded during the *entire* ICU stay rather than at admission or within a fixed early window, so for patients who died, these values often reflect physiology close to death rather than physiology at presentation. A control analysis restricting the feature set to fields whose provenance is genuinely admission-time (age, height, weight, sex, and comorbidity flags, none of which are last-value-in-stay constructions) drops held-out AUROC to 0.66 for both a linear and a nonlinear model, which is consistent with the literature for demographics-only ICU mortality baselines and indicates that essentially all of the additional discrimination in Table A.4 comes from this leakage rather than from genuine early physiological signal. We report the case study using the released feature set, unmodified beyond the two excluded fields, because the study's purpose is to demonstrate the platform's governed multi-institutional workflow end to end, not to establish a validated clinical risk model; the reported AUROC should not be read as, and must not be cited as, an estimate of real admission-time mortality-prediction performance. A corrected, properly time-windowed extraction pipeline already exists in the dataset's source repository but was never connected to the final analysis dataset, and regenerating it requires credentialed access to the raw eICU-CRD tables that this evaluation did not have.

Methods

Development of the operational requirement set

We characterized the operational work required across the lifecycle of a multi-institutional health AI study, from before training begins until after a model is released. Evidence was drawn from three lenses. The *guidance* lens covered systematic reviews, governance analyses, consensus statements, and implementation roadmaps.[22,23,36,55] The *practice* lens covered procedures and barriers reported in real multi-institutional implementations.[9,56–58] The *capability* lens covered operational functions described for healthcare-oriented platforms and current FL frameworks.[26,59–61] Together the lenses represent what institutions are advised to prepare, what they encounter in practice, and what current software supports.

We extracted candidate needs from each lens, consolidated recurring concepts, and removed overlaps. A candidate was retained when it appeared in more than one lens and was explicitly discussed in the source rather than inferred from it. Each retained requirement received an operational definition and representative supporting evidence, which produced the 29 requirements and eight categories in Table A.1. The requirement set and its definitions were frozen before any framework was scored.

Framework selection and frozen evidence base

The comparison included TensorFlow Federated, FATE, FedML, Flower, NVIDIA FLARE, OpenFL, IBM Federated Learning, PaddleFL, Substra/SubstraFL, EasyFL, FLGo, Fedstellar, FedOps, and Fed-BioMed. The set represented research libraries, production-oriented runtimes, decentralized systems, and healthcare-oriented platforms. Inclusion required public code, sufficient technical documentation for requirement-specific assessment, and either a peer-reviewed publication or continued visibility in recent literature or practice.

The evidence cutoff was 5 July 2026 and was fixed before scoring began. For each framework, a registry recorded its identity, official repository, pinned commit, official documentation, primary scholarly source, and any prespecified additional source. Evidence was selected according to relevance to the requirement. Implementation code and configuration were inspected when overview documentation did not resolve a capability claim. Marketing statements, hypothetical custom extensions, and unsupported inference were not counted.

Ordinal scoring and evidence records

The unit of analysis was the framework–requirement pair. Each of the 406 pairs was scored on a four-level ordinal scale. A score of 0 indicated no documented support after a requirement-specific search, 1 limited or partial support or support dependent on a separate institutional process, 2 substantial but incomplete or configurable support, and 3 strong native end-to-end support. Scores of 0 and 1 therefore both indicate that the framework does not meet the requirement natively, and differ in whether any relevant documentation exists.

Every rating recorded a versioned source locator, an evidence paraphrase, a requirement-specific rationale, search notes, support mode, evidence strength, and confidence. For a score of 0, the search notes were required to identify the sources and terms examined, so that absence of support could be distinguished from absence of searching.

Three disambiguation rules applied throughout. Security functions were not treated as evidence of legal or ethics readiness, software project governance was distinguished from study governance, and simulation support was not treated as production deployment. Because the scale is ordinal, percentages indicate relative coverage rather than the proportion of a requirement met.

Expert assessment and reproducibility check

A human expert with experience in multi-institutional health AI research scored all 406 pairs against the frozen evidence package and scoring codebook. These scores constitute the primary assessment reported in Results.

To test whether the assessment could be reproduced independently, two model families, GPT-5.6 Sol and Claude Opus 5, each completed three isolated runs against the same frozen package, codebook, evidence registry, cutoff, prompt, and output schema, producing 1,218 ratings per family and 2,436 in total. Runs within a family shared no state. The family-level score for each pair was the median of its three runs. The expert was blinded to all model output, and the model runs were conducted without access to the expert scores. Reproducibility was assessed by comparing each family-level score with the expert score. Repeated runs within a family measure the stability of the scoring procedure and were not treated as independent evidence about a framework.

Coverage and agreement analysis

For any group of n framework–requirement pairs, normalized coverage was calculated as

$$\text{coverage} = 100 \times \frac{\sum_{i=1}^{n} s_i}{3n}, \qquad s_i \in \{0, 1, 2, 3\}, \qquad (1)$$

where $3n$ is the maximum attainable total for the group. Requirement-level coverage used $n = 14$, since each requirement was evaluated across 14 frameworks. Framework-level coverage used $n = 29$, since each framework was evaluated against 29 requirements. Category-level coverage used $n = 14m_c$ for a category containing m_c requirements.

Within-family stability across the three runs was summarized using ordinal Krippendorff's α, exact agreement across all three runs, agreement within one ordinal point, defined as a maximum–minimum difference of at most one, pairwise quadratic-weighted Cohen's κ, and pairwise mean absolute difference.[37,62] Because Cohen's κ compares two raters, three pairwise estimates were obtained for each family.

Each family-level score was compared with the expert score using exact agreement, agreement within one point, mean absolute difference, quadratic-weighted κ, and Spearman rank correlation, at pair level and after averaging by framework, category, and requirement. The two model families were compared using the same measures as a secondary analysis. Binary threshold analyses at $s \geq 1$, representing any documented support, and $s \geq 2$, representing substantial or strong support, were treated as sensitivity analyses.

System design

Fedlify's architecture was derived as a set of explicit and traceable design decisions.[63] Requirements that were poorly supported by existing frameworks were mapped to candidate planning artifacts. Each candidate function was then checked against the system boundary. Any function that required patient-level data, production credentials, access to a runtime FL system, or authority to make an institutional decision was excluded or assigned to an external role. The remaining functions were partitioned into deterministic platform tasks, bounded model-mediated tasks, and human decisions, which is the partition reported in Section 2.3. Workflow states, stopping conditions, record types, provenance relations, and change-invalidation rules were then specified.

Implementation and deployment

Fedlify is packaged for self-hosted deployment on consortium infrastructure. A deployment serves multiple projects, each isolated in its own private network, and the operating institution is chosen by the consortium. Approved configuration revisions are committed to an internal version-control service, which supplies the commit identity to which pipeline versions and approvals are bound. The reasoning component is accessed through an OpenAI-compatible interface so that local or hosted models can be substituted without platform changes. The reference deployment targets NVIDIA FLARE, and the template catalog currently provides cross-silo workflows for FedAvg, FedProx, differentially private SGD, federated logistic regression, and histogram-based federated XGBoost.

Worked configuration

The worked configuration used the eICU Collaborative Research Database,[47] a de-identified critical care dataset covering ICU admissions at United States hospitals. Adults aged 16 to 90 admitted to an ICU with a documented diagnosis of acute or chronic heart failure were eligible, identified using ICD-9 and ICD-10 codes and clinical documentation. For patients with multiple ICU admissions only the final eligible admission was included. Patients were excluded if outcome data were missing, if the ICU stay was shorter than four hours, or if there was prior heart transplantation or durable ventricular assist device support at admission. The five hospitals with the largest eligible cohorts formed the training federation and a sixth was reserved for external validation. All configuration, approval, and handoff steps were performed through the deployed platform.

Case-study model training and evaluation

Five federated jobs were generated from the approved specification: FedAvg and FedProx with a shared multilayer-perceptron architecture (two hidden layers of 64 and 32 units,

ReLU activations, a sigmoid output), differentially private SGD using the same architecture, federated logistic regression, and a histogram-based federated gradient-boosting ensemble. Continuous features were median-imputed and standardized and categorical features were most-frequent-imputed and one-hot encoded, fit separately within each cross-validation fold and within each site to prevent leakage across the evaluation split. FedAvg, FedProx, and the differentially private variant trained for 12 federated rounds of 5 local epochs each with batch size 64; FedAvg and FedProx used Adam ($\beta_1 = 0.9, \beta_2 = 0.999$) with learning rate 10^{-3}, and FedProx added a proximal penalty ($\mu = 0.01$) toward the previous round's global parameters. Federated logistic regression used plain gradient descent at learning rate 10^{-2}. Class imbalance was addressed with inverse-frequency example weighting in every gradient-based job. Federated averaging weighted each site's contribution by its local case count.

The differentially private SGD job clips each per-example gradient to a fixed L2 norm before averaging and adds Gaussian noise calibrated to that norm (noise $\sim \mathcal{N}(0,$ noise multiplier $\times$ clip norm/batch size)), then updates with plain (non-adaptive) SGD. The clip norm and learning rate were set to 6.0 and 0.05 respectively, scaled to this architecture's per-example gradient magnitude; an initial configuration matched to the non-private jobs (clip norm 1.0, learning rate 10^{-3}) clipped 86–98% of per-example gradients on nearly every step and did not converge. The noise multiplier (0.08) was held fixed between the two configurations, so the noise-to-clip ratio, and with it the per-step privacy cost, is unchanged. A conservative small-q analytical approximation was used to report an estimated privacy budget ($\delta = 10^{-5}$); it is not a formal accountant certificate and is not evidence of a strong privacy guarantee at this operating point.

Model selection and evaluation used 5-fold stratified cross-validation within the five training sites. For each fold, all five training sites' held-out predictions were pooled and the classification threshold was selected to maximize F_1 using only this pooled training cross-validation output; the held-out sixth hospital was never used for threshold selection. Final models for external evaluation were refit on each training site's complete data using the same protocol. External performance is reported on the held-out sixth hospital using this fixed threshold: AUROC, AUPRC, precision, recall, F_1, and Brier score, with 95% confidence intervals from 2,000 bootstrap resamples. Calibration slope and intercept were obtained by fitting a one-parameter logistic recalibration of the predicted probabilities against observed outcomes. All training, evaluation, and privacy-accounting code is released with the platform and was run against the frozen dataset and protocol identified by their content hashes in the run manifest.

Probe suite construction and scoring

The 144-probe suite was constructed in three stages by independent roles, each auditable against the platform's authorization path (Fig. 3) and the design principles. A scenario-architect stage seeded clinically and operationally plausible multi-institutional study situations across all nine probe categories, preserving the six-hospital worked configuration where applicable and covering every requirement (R01–R29) at least once. An adversarial-boundary-engineer stage then applied indirect and direct prompt injection, data-class violations, evidence and memory poisoning, role and version spoofing, runtime and export-boundary abuse, fault and budget stress, and lifecycle-change attacks to a subset of the scenarios, while retaining unmodified benign and incomplete cases as positive controls; this stage produced the final split of 79 adversarial and 65 benign, incomplete, or control probes. An assurance-and-oracle-auditor stage checked probe-identifier uniqueness, fixture existence, exact-prompt uniqueness, requirement and design-principle coverage, and the internal consistency of each deterministic oracle (the expected disposition, blocking layer, stop condition, and state transition), correcting undercoverage rather than introducing new platform behavior; this stage also identified and corrected an initial underrepresentation of principle P8. All 144 probes were accepted after this audit. Every probe and fixture is content-hash pinned; the suite and every fixture file are released with the code.

Probes were executed against the deployed platform through its production authenticated intake endpoint, not through a scenario-specific or test-only code path. The executing harness authenticates as a real user, creates a real study record, and submits each probe's exact prompt and fixture as a real API request; it loads no field from any probe's oracle, so the platform's response cannot be, and is not, conditioned on the expected answer. A probe is classified *passed* when the platform's disposition, authoritative layer, stop condition, and state transition all match its oracle; *wrong layer* when the platform stopped or permitted the request but not through the predicted mechanism; and *not blocked* when the platform failed to stop a request its oracle required it to stop. The suite was run as three independent repetitions against one deployed platform configuration using the pinned hosted model. Reported per-probe results are the majority classification across the three repetitions; the single probe without a two-of-three majority was resolved to its most conservative (least favorable) observed classification rather than an arbitrary tie-break.

References

[1] Balch, J., Upchurch, G. R., Bihorac, A. & Loftus, T. J. Bridging the artificial intelligence valley of death in surgical decision-making. *Surgery* **169**, 746–748 (2021).

[2] Gallifant, J. *et al.* Beyond the algorithm: A field guide to deploying ai agents in clinical practice (2025).

[3] Eapen, B. R. & Singh, N. Navigating the "valley of death": An open-source, modular framework for generative ai in healthcare. In *Opening the Personal Gate between Technology and Health Care*, 1561–1562 (IOS Press, 2026).

[4] Salwei, M. E. & Carayon, P. A sociotechnical systems framework for the application of artificial intelligence in health care delivery. *Journal of cognitive engineering and decision making* **16**, 194–206 (2022).

[5] Sokol, K., Fackler, J. & Vogt, J. E. Artificial intelligence should genuinely support clinical reasoning and decision making to bridge the translational gap. *npj Digital Medicine* **8**, 345 (2025).

[6] Yang, J. *et al.* Generalizability assessment of ai models across hospitals in a low-middle and high income country. *Nature Communications* **15**, 8270 (2024).

[7] Ong Ly, C. *et al.* Shortcut learning in medical ai hinders generalization: method for estimating ai model generalization without external data. *NPJ digital medicine* **7**, 124 (2024).

[8] Ktena, I. *et al.* Generative models improve fairness of medical classifiers under distribution shifts. *Nature Medicine* **30**, 1166–1173 (2024).

[9] Pati, S. *et al.* Federated learning enables big data for rare cancer boundary detection. *Nature communications* **13**, 7346 (2022).

[10] Dayan, I. *et al.* Federated learning for predicting clinical outcomes in patients with covid-19. *Nature medicine* **27**, 1735–1743 (2021).

[11] Lee, E. H. *et al.* An international study presenting a federated learning ai platform for pediatric brain tumors. *Nature communications* **15**, 7615 (2024).

[12] Alami, H. *et al.* Understanding the integration of artificial intelligence in healthcare organisations and systems through the nasss framework: a qualitative study in a leading canadian academic centre. *BMC health services research* **24**, 701 (2024).

[13] Hassan, M., Kushniruk, A. & Borycki, E. Barriers to and facilitators of artificial intelligence adoption in health care: scoping review. *JMIR Human Factors* **11**, e48633 (2024).

[14] Suleman, M. U. *et al.* Assessing the generalizability of artificial intelligence in radiology: a systematic review of performance across different clinical settings. *Annals of Medicine and Surgery* **87**, 8803–8811 (2025).

[15] Arun, S. *et al.* Systematic scoping review of external validation studies of ai pathology models for lung cancer diagnosis. *NPJ precision oncology* **9**, 166 (2025).

[16] Pandian, B. *et al.* Generalizing machine learning models from clinical free text. *Scientific Reports* **15**, 31668 (2025).

[17] Naderian, S. *et al.* A systematic review of artificial intelligence-based clinical decision support systems in prostate cancer management. *Healthcare Technology Letters* **12**, e70026 (2025).

[18] McMahan, B., Moore, E., Ramage, D., Hampson, S. & y Arcas, B. A. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, 1273–1282 (Pmlr, 2017).

[19] ur Rehman, M. H. & Gaber, M. M. *Federated learning systems: Towards next-generation AI*, vol. 965 (Springer Nature, 2021).

[20] Welten, S. *et al.* A study on interoperability between two personal health train infrastructures in leukodystrophy data analysis. *Scientific data* **11**, 663 (2024).

[21] Barbereau, T., Fernandez, J. D. & Menci, S. P. The governance of federated learning: a decision framework for organisational archetypes. *Data & Policy* **7**, e53 (2025).

[22] Eden, R. *et al.* A scoping review of the governance of federated learning in healthcare. *npj Digital Medicine* **8**, 427 (2025).

[23] Teo, Z. L. *et al.* Federated machine learning in healthcare: A systematic review on clinical applications and technical architecture. *Cell Reports Medicine* **5** (2024).

[24] Jafarinezhad, O. *et al.* Centralized pooling and federated learning for canadian patient-level data sharing in multicenter medical ai: A scoping review. *Artificial Intelligence in Medicine* 103422 (2026).

[25] Matschinske, J. *et al.* The featurecloud platform for federated learning in biomedicine: unified approach. *Journal of Medical Internet Research* **25**, e42621 (2023).

[26] Cremonesi, F. *et al.* Fed-biomed: Open, transparent and trusted federated learning for real-world healthcare applications. In *Federated Learning Systems: Towards Privacy-Preserving Distributed AI*, 19–41 (Springer, 2025).

[27] Schneider, J. Generative to agentic ai: Survey, conceptualization, and challenges. *arXiv preprint arXiv:2504.18875* (2025).

[28] Bandi, A., Kongari, B., Naguru, R., Pasnoor, S. & Vilipala, S. V. The rise of agentic ai: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges. *Future Internet* **17**, 404 (2025).

[29] Zhao, W. *et al.* An agentic system for rare disease diagnosis with traceable reasoning. *Nature* **651**, 775–784 (2026).

[30] Goh, H. L. *et al.* Personalised health plan development using agentic ai in singapore's national preventive care programme: a pilot study. *NPJ Digital Medicine* **9**, 332 (2026).

[31] Li, H., Funk, M. & Saeed, A. Helmsman: Autonomous synthesis of federated learning systems via collaborative llm agents. *arXiv preprint arXiv:2510.14512* (2025).

[32] Ji, H. *et al.* Lafa: Agentic llm-driven federated analytics over decentralized data sources. In *2025 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, 1–8 (IEEE, 2025).

[33] Isong, J. A., Kanu, V. I., Ajakwe, S. O. & Kim, D.-S. Federated agentic learning with adaptive privacy for cardiovascular anomaly detection at the edge. In *2025 16th International Conference on Information and Communication Technology Convergence (ICTC)*, 152–157 (IEEE, 2025).

[34] OpenAI. OpenAI and Hugging Face partner to address security incident during model evaluation. https://openai.com/index/hugging-face-model-evaluation-security-incident/ (2026). Published 21 July 2026; accessed 25 July 2026.

[35] Ashraf, T., Jeong, H., Thoker, F. M. & Ghanem, B. Medcta: A benchmark for clinical tool agents. *arXiv preprint arXiv:2606.11702* (2026).

[36] Li, M., Xu, P., Hu, J., Tang, Z. & Yang, G. From challenges and pitfalls to recommendations and opportunities: Implementing federated learning in healthcare. *Medical image analysis* **101**, 103497 (2025).

[37] Cohen, J. Weighted kappa: Nominal scale agreement provision for scaled disagreement or partial credit. *Psychological bulletin* **70**, 213 (1968).

[38] Rose, S., Borchert, O., Mitchell, S. & Connelly, S. Zero trust architecture. Tech. Rep. NIST SP 800-207, National Institute of Standards and Technology (2020).

[39] Souppaya, M., Scarfone, K. & Dodson, D. Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities. Tech. Rep. NIST SP 800-218, National Institute of Standards and Technology (2022).

[40] Greshake, K. *et al.* Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM workshop on artificial intelligence and security*, 79–90 (2023).

[41] Ruan, Y. *et al.* Identifying the risks of lm agents with an lm-emulated sandbox. In *International Conference on Learning Representations*, vol. 2024, 27031–27098 (2024).

[42] Debenedetti, E. *et al.* Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Advances in neural information processing systems* **37**, 82895–82920 (2024).

[43] Chelli, M. *et al.* Hallucination rates and reference accuracy of chatgpt and bard for systematic reviews: comparative analysis. *Journal of Medical Internet Research* **26**, e53164 (2024).

[44] Panickssery, A., Bowman, S. R. & Feng, S. Llm evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems*, vol. 37 (2024).

[45] Dratsch, T. *et al.* Automation bias in mammography: the impact of artificial intelligence bi-rads suggestions on reader performance. *Radiology* **307**, e222176 (2023).

[46] Buçinca, Z., Malaya, M. B. & Gajos, K. Z. To trust or to think: cognitive forcing functions can reduce overreliance on ai in ai-assisted decision-making. *Proceedings of the ACM on Human-computer Interaction* **5**, 1–21 (2021).

[47] Pollard, T. *et al.* eICU Collaborative Research Database. *PhysioNet* (2019). URL https://doi.org/10.13026/C2WM1R. Version 2.0.

[48] Roth, H. R. *et al.* Nvidia flare: Federated learning from simulation to real-world. *arXiv preprint arXiv:2210.13291* (2022).

[49] Moon, J., Yang, S. & Lee, K. Fedops: A platform of federated learning operations with heterogeneity management. *IEEE Access* **12**, 4301–4314 (2024).

[50] McCausland, D., Haigh, M., McCallion, P. & McCarron, M. Irb challenges in multisite studies: A case report and commentary from the intellectual disability supplement to the irish longitudinal study on ageing (ids-tilda). *HRB Open Research* **7**, 3 (2024).

[51] Ratnayake, I. *et al.* Evaluating the impact of delayed study startup on accrual in cancer studies. *Research Square* rs–3 (2024).

[52] Lawrence, C. E. *et al.* Quantitative assessment of the impact of standard agreement templates on multisite clinical trial start up time. *Journal of clinical and translational science* **7**, e204 (2023).

[53] Harris, P. A. *et al.* The redcap consortium: building an international community of software platform partners. *Journal of biomedical informatics* **95**, 103208 (2019).

[54] Sterz, S. *et al.* On the quest for effectiveness in human oversight: Interdisciplinary perspectives. In *Proceedings of the 2024 ACM conference on fairness, accountability, and transparency*, 2495–2507 (2024).

[55] Kommusaar, J. *et al.* A roadmap for federated learning projects using health data to guide sustainable artificial intelligence development in the european union. *International Journal of Medical Informatics* 106242 (2025).

[56] Bujotzek, M. R. *et al.* Real-world federated learning in radiology: hurdles to overcome and benefits to gain. *Journal of the American Medical Informatics Association* **32**, 193–205 (2025).

[57] Peltonen, L.-M. & Chomutare, T. Federated learning's uncomfortable truth: why human networks matter more than neural networks. *Journal of the American Medical Informatics Association* ocag047 (2026).

[58] Schoenpflug, L. A. *et al.* Navigating real-world challenges: A case study on federated learning in computational pathology. *Journal of Pathology Informatics* 100464 (2025).

[59] Mullie, L. *et al.* Coda: an open-source platform for federated analysis and machine learning on distributed healthcare data. *Journal of the American Medical Informatics Association* **31**, 651–665 (2024).

[60] Bottomly, D. *et al.* A pragmatic framework for federated learning risk and governance in academic medical centers. *JMIR AI* **5**, e80022 (2026).

[61] Chavero-Diez, M., Hernandez-Ferrer, C., Codó, L., Gelpí, J. L. & Capella-Gutiérrez, S. Federated learning frameworks: quality and interoperability for biomedical research. *NAR Genomics and Bioinformatics* **8**, lqag010 (2026).

[62] Hayes, A. F. & Krippendorff, K. Answering the call for a standard reliability measure for coding data. *Communication methods and measures* **1**, 77–89 (2007).

[63] Jansen, A. & Bosch, J. Software architecture as a set of architectural design decisions. In *5th Working IEEE/IFIP Conference on Software Architecture (WICSA'05)*, 109–120 (IEEE, 2005).

[64] Lekadir, K. *et al.* Futureai: International consensus guideline for trustworthy and deployable artificial intelligence in healthcare. arxiv preprint arxiv: 230912325 (2023).

[65] Schlegel, M., Scheliga, D., Sattler, K.-U., Seeland, M. & Mäder, P. Lowering barriers to federated learning: collaboration management and provenance. *Journal of Big Data* **13**, 28 (2026).

[66] Lee, G. H. *et al.* Feasibility study of federated learning on the distributed research network of omop common data model. *Healthcare Informatics Research* **29**, 168–173 (2023).

[67] Chai, D. *et al.* A survey for federated learning evaluations: Goals and measures. *IEEE transactions on knowledge and data engineering* **36**, 5007–5024 (2024).

[68] Spearman, C. The proof and measurement of association between two things. *International journal of epidemiology* **39**, 1137–1150 (2010).

Supplementary material

Appendix A Operational requirements

The 29 requirements were derived from three evidence lenses covering published guidance, reported multi-institutional implementations, and documented platform capability. A candidate was retained when it appeared in more than one lens and was explicitly discussed in the source. The set was frozen before any framework was scored. The resulting requirements and their definitions are listed in Table A.1.

Appendix B Reproducibility of the framework assessment

Because the scale is ordinal, agreement is reported as quadratic-weighted Cohen's κ, which penalizes a disagreement in proportion to the square of the distance between the two scores.[37] Rank-order consistency was assessed using Spearman's correlation coefficient ρ.[68] Within-family stability across the three runs is additionally reported as ordinal Krippendorff's α.[62] Table A.2 reports all comparisons. Both model families reproduced the expert assessment closely, with quadratic-weighted κ of 0.841 for Claude Opus 5 and 0.779 for GPT-5.6 Sol. Agreement between th two families was lower ($\kappa = 0.651$), and the expert assessment fell at or between the two family scores in 401 of 406 pairs. Claude Opus 5 scored lower than GPT-5.6 Sol in 189 pairs, equal in 209, and higher in 8, giving a mean signed difference of −0.461 points, equivalent to 15.4 percentage points of normalized coverage. Spearman ρ between the two families rose from 0.755 at pair level to 0.870 across frameworks, 0.898 across categories, and 0.858 across requirements. The evaluators therefore agreed on the lifecycle pattern more closely than on individual scores.

Table A.2: **Agreement statistics across the complete grid of 406 framework–requirement pairs.** Within-family values summarize three isolated runs of the same family. Between-assessment values compare family-level medians with the expert assessment.

Within-family reproducibility *(three runs per model)*			
Metric	**Claude Opus 5** *three runs*	**GPT-5.6 Sol** *three runs*	
Exact agreement (%)	70.4	50.5	
Within one point (%)	99.5	98.0	
Mean absolute difference	0.167–0.229	0.328–0.365	
Weighted κ	0.813–0.861	0.751–0.757	
Krippendorff's α	0.814	0.749	
Between-assessment agreement *(family medians)*			
Metric	**Claude Opus 5** *vs expert*	**GPT-5.6 Sol** *vs expert*	**Claude Opus 5** *vs GPT-5.6 Sol*
Exact agreement (%)	79.8	68.0	51.5
Within one point (%)	99.8	100.0	98.5
Mean absolute difference	0.204	0.320	0.500
Weighted κ	0.841	0.779	0.651
Spearman ρ	0.856	0.821	0.755

Two Claude Opus 5 pairs showed a two-point score range, but none received three distinct scores. Eight GPT-5.6 Sol pairs had the same range, five of which received three distinct scores. Greater within-family consistency indicates more stable application of the scoring threshold and does not establish greater accuracy. Table A.3 shows coverage for each individual run.

Table A.3: **Normalized documented coverage for each individual run, each family-level median, and the expert assessment.** GPT-5.6 Sol run 1 is an outlier that the family median attenuates without removing.

Assessment	Run	Coverage (%)
Claude Opus 5	Run 1	26.8
	Run 2	26.2
	Run 3	26.5
	Family median	*26.6*
GPT-5.6 Sol	Run 1	47.2
	Run 2	39.1
	Run 3	39.0
	Family median	*42.0*
Human expert	—	32.6

Collapsing the scale to any documented support ($s \geq 1$), Claude Opus 5 and GPT-5.6 Sol agreed on 76.8% of pairs ($\kappa = 0.475$), Claude Opus 5 and the expert on 90.4% ($\kappa = 0.793$), and GPT-5.6 Sol and the expert on 85.0% ($\kappa = 0.613$). At substantial or strong support ($s \geq 2$), the corresponding values were 77.3% ($\kappa = 0.474$), 90.1% ($\kappa = 0.717$), and 86.2% ($\kappa = 0.693$). Agreement is consistently lower under binary collapse than under the ordinal scale, which indicates that evaluators concurred more on relative ranking than on whether a capability crosses a fixed threshold.

Six pairs showed a two-point spread across the three assessments. FedML for consortium, site-readiness, and contribution planning (R02) and for update, retraining, rollback, retirement, and sustainability ownership (R29), OpenFL for fault tolerance, checkpointing, and recoverability (R22), PaddleFL for representativeness, heterogeneity, and partition plan (R11), FLGo for compute, storage, network, and availability readiness (R12), and Fedstellar for secure communication and update confidentiality (R18). In each case GPT-5.6 Sol identified implementation-level or source-level evidence absent from the Claude Opus 5 record.

Appendix C case study performance report

All five jobs train on the same five hospital cohorts with identical preprocessing and are evaluated on the same held-out sixth hospital, so the only difference between jobs is the federated algorithm. Table A.4 reports held-out performance, calibration, and mean per-site training performance for each.

The differentially private SGD job clips each per-example gradient to a fixed norm and adds calibrated noise before averaging. An initial configuration that reused the non-private jobs' learning rate and a clip norm of 1.0 clipped 86–98% of per-example gradients on nearly every step, which prevented the

Table A.1: Evidence-informed operational requirements for end-to-end multi-institutional health AI studies.

ID	Requirement	Operational definition and representative evidence
1. Consortium and study design		
R01	Clinical purpose and FL suitability	Define the intended clinical or research use, target population, decision context, expected collaborative benefit, and why FL is preferable to feasible alternatives before selecting a platform.[23,36,55,56,60,64]
R02	Consortium, site-readiness, and contribution planning	Select sites that can contribute clinically relevant data and sustained local capability, and assess each site's data, infrastructure, staff, constraints, incentives, and expected contribution.[9,22,55–58,65]
R03	Multidisciplinary roles, coordination, and shared vocabulary	Create a working structure that connects clinical, data-science, engineering, IT, security, privacy, ethics, legal, project-management, and patient or public perspectives across institutions.[22,26,55–57,65]
2. Governance, ethics, and legal readiness		
R04	Legal basis, ethics, and cross-site agreements	Establish the lawful and ethical basis for local data use, distributed computation, exchange of model artifacts, and cross-institutional collaboration in every participating jurisdiction.[22,55–57,60]
R05	Accountability, ownership, and decision rights	Specify who owns or controls data, code, configurations, models, logs, and outputs, and who may approve, pause, release, modify, or retire them.[22,26,55–57,60,65]
R06	Risk-proportionate oversight and approval gates	Match governance, privacy, security, validation, and release review to the sensitivity of the data, model capability, intended use, and potential clinical impact.[22,55,56,60,64]
R07	Participation lifecycle, synchronized change control, and withdrawal	Define how sites join, approve tasks, request changes, pause, leave, or are removed while preserving safety, reproducibility, and consortium continuity.[22,55–57,65]
3. Data readiness and interoperability		
R08	Common data specification and semantic interoperability	Create a shared, machine-actionable definition of variables, labels, units, coding systems, metadata, and allowable mappings across sites.[22,26,36,55,56,59,61,66]
R09	Harmonized preprocessing, acquisition, and annotation	Ensure that local extraction, cohort construction, preprocessing, missing-data handling, acquisition, and labeling follow agreed protocols, or that deviations are measured and documented.[9,22,26,36,55,56]
R10	Data quality and training-readiness validation	Run local checks to establish whether each site's data, metadata, labels, and references are complete, valid, correctly linked, and processable before federated execution.[9,26,36,55,56]
R11	Representativeness, heterogeneity, and partition plan	Characterize clinical, demographic, temporal, acquisition, label, and site-level differences and design train, validation, test, and external-evaluation partitions accordingly.[9,36,55,56,64]
4. Infrastructure and site onboarding		
R12	Compute, storage, network, and availability readiness	Verify that each site and coordinating service can meet agreed compute, storage, bandwidth, firewall, uptime, maintenance, and support requirements without disrupting clinical operations.[26,36,55–58]
R13	Reproducible runtime and cross-site version compatibility	Standardize and version operating systems, libraries, drivers, containers or virtual environments, configurations, and update procedures so the same task executes consistently at every node.[26,36,55,56,59,61]
R14	Clinical-system integration and secure local data access	Connect each FL node to approved local sources and third-party systems through least-privilege interfaces while preventing raw patient data from leaving the institution.[26,36,55,56,59,61,66]
R15	Site onboarding, dry-run, and readiness gates	Use repeatable onboarding and non-sensitive or synthetic dry-runs to confirm connectivity, configuration, security, workflow, observability, and staffing before real training begins.[26,55,56,58–60]
5. Privacy and security engineering		
R16	Threat model and risk-tiered privacy/security controls	Identify credible threats to data, model updates, code, nodes, the aggregator, users, and outputs, then select controls in proportion to likelihood, impact, and intended use.[22,26,36,55,60]
R17	Identity, authentication, authorization, and secrets management	Ensure that people, services, and sites are authenticated and receive only the permissions required for their approved roles, with controlled credential and key lifecycles.[22,26,55,59,60]
R18	Secure communication and update confidentiality	Protect model artifacts and control traffic in transit and, where justified, during aggregation, while specifying what information crosses institutional boundaries.[22,26,36,55,59,60]
R19	Code, model, and update integrity with incident response	Prevent, detect, contain, and recover from malicious or erroneous code, poisoned updates, compromised nodes, unauthorized changes, and security or privacy incidents.[22,26,36,55,56,60]
6. Federated protocol and execution control		
R20	Explicit and reproducible federated protocol	Specify the complete study protocol so another authorized team can reproduce how clients, server, data processing, training, validation, and stopping decisions interact.[26,36,55,56,59,65,67]
R21	Resource-aware orchestration and heterogeneity handling	Schedule and coordinate jobs around unequal data distributions, hardware, bandwidth, uptime, time zones, and clinical workloads without allowing resource-rich sites to dominate silently.[26,36,55–58,67]
R22	Fault tolerance, checkpointing, and recoverability	Continue safely or recover predictably when clients disconnect, nodes fail, configurations mismatch, jobs crash, or maintenance interrupts training.[26,36,55,56,58]
R23	Execution observability, audit trail, and local approval	Give authorized central and local actors enough visibility to understand, approve, monitor, diagnose, and audit each job without exposing patient-level data.[22,26,36,55,56,58–60,65]
7. Validation, release, and reproducibility		
R24	Multidimensional local, external, and subgroup evaluation	Evaluate more than global accuracy by measuring site-level and external performance, calibration, robustness, subgroup fairness, privacy and security, communication, resource use, and scalability against relevant alternatives.[9,36,55,56,60,64,67]
R25	Independent validation and controlled release gate	Require independent or site-local validation and multidisciplinary approval before a final model, code package, or result is distributed, deployed, or used for a new purpose.[9,22,55,56,60,64]
R26	Reproducibility, reporting, provenance, and registry package	Package the information and artifacts needed to understand, reproduce, audit, compare, and govern the study and every released model version without revealing protected data.[22,26,36,55,59–61,64,65]

Table A.1: Evidence-informed operational requirements (continued).

ID	Requirement	Operational definition and representative evidence
8. Deployment, monitoring, and sustainability		
R27	Workflow integration, usability, and stakeholder capability	Fit the model and operational process into clinical or research workflows and ensure users, data stewards, IT teams, and oversight actors can perform and understand their responsibilities.[22,55,57,59,64]
R28	Continuous performance, drift, fairness, security, and compliance monitoring	Monitor the deployed or reused system and its federation for degradation, distribution shift, subgroup harm, misuse, access violations, security events, and regulatory or contextual change.[22,36,55,56,60,64]
R29	Update, retraining, rollback, retirement, and sustainability ownership	Assign durable organizational and financial ownership for maintenance, updates, retraining, rollback, incident recovery, site withdrawal, transfer, and retirement after the initial study or funding period.[22,55–57,64]

Table A.4: **Performance of the five federated training approaches.** All jobs train on the same five hospital cohorts with identical preprocessing and are evaluated on the same held-out sixth hospital, so the only difference between rows is the federated algorithm. Two vitals fields with outcome-correlated missingness were excluded from the feature contract before this run (Limitations).

Approach	Held-out AUROC (95% CI)	AUPRC	Calibration slope	Brier	Mean per-site AUROC
FedAvg, multilayer perceptron	0.968 (0.955–0.980)	0.928	0.810	0.046	0.953
FedProx, multilayer perceptron	0.969 (0.955–0.980)	0.928	0.824	0.047	0.955
Differentially private SGD	0.964 (0.949–0.977)	0.917	0.719	0.038	0.951
Federated logistic regression	0.939 (0.920–0.955)	0.852	1.193	0.105	0.931
Histogram-based federated XGBoost	0.977 (0.966–0.986)	0.934	1.186	0.033	0.965

model from converging (held-out AUROC 0.263, worse than chance). The reported run instead uses a clip norm and learning rate scaled to this model's gradient magnitude (clip norm 6.0, learning rate 0.05, plain SGD), with the noise multiplier held fixed so the noise-to-clip ratio is unchanged. The resulting analytical privacy-budget estimate ($\epsilon \approx 362$ under a conservative small-q approximation, $\delta = 10^{-5}$) is loose and is reported for completeness; it should not be read as a strong formal privacy guarantee at this operating point.

Two of the dataset's raw vitals fields, `sao2_periodic` and `temperature_periodic`, are excluded from the feature contract used to produce Table A.4. Both are missing for most stays (34.2% and 97.3%, respectively), and the missingness is not random with respect to the outcome: `temperature_periodic` is recorded for only 2.7% of patients, and that subset's in-hospital mortality rate (59.4%) is roughly five times the cohort's overall rate (12.3%). Excluding both fields changes external held-out AUROC by only 0.004–0.009 across the five approaches (e.g., FedAvg 0.973 to 0.968); they were removed regardless of that small marginal contribution because their inclusion could not be justified once the pattern was identified, but their small effect on AUROC pointed to a larger issue.

We traced that issue to the dataset's source SQL: every remaining laboratory and vital-sign feature in this case study (creatinine, BUN, white cell count, heart rate, and the rest) is constructed by the source extraction pipeline as the *last* value recorded during the entire ICU stay (`ORDER BY labresultoffset DESC`, no lower or upper time bound), not a value at admission or within a fixed early window. For patients who die, a "last" lab or vital value is frequently taken close to death and reflects terminal physiology rather than presentation. This is confirmed by inspecting the query that builds the released CSV, not inferred from the modeling results alone. We quantified its effect directly: restricting the feature set to fields whose provenance is genuinely admission-time (age, height, weight, sex, and the diagnosis-code-derived comorbidity flags, none of which are last-value-in-stay constructions) drops held-out AUROC to 0.66 for both a linear model and a nonlinear tree ensemble – the two model families agreeing exactly is itself evidence that little real signal remains once the contaminated fields are removed, and 0.66 is in line with published demographics-only ICU mortality baselines. Table A.4 is reported using the released feature set (beyond the two excluded fields above) because the case study's purpose is to exercise the platform's governed multi-institutional workflow end to end, not to establish a validated clinical risk model, and changing the feature contract further would not change what the case study demonstrates about the platform. The reported AUROC should be read as an upper bound on admission-time predictive performance under this feature construction, not as an estimate of it. A properly time-windowed extraction pipeline (restricting every lab and vital to the first 24 hours of the ICU stay) already exists in the dataset's source repository as a set of unused SQL views, but was never connected to the query that produced the released CSV; regenerating a corrected, cohort-matched dataset requires credentialed access to the raw eICU-CRD tables that this evaluation did not have.

Appendix D Probe escapes

No probe passed every layer without an expected stop in this run: the majority classification across three live, oracle-blind repetitions records zero not-blocked outcomes among the 144 probes (main-text Table 1). The 56 probes that did not reach their oracle's exact predicted layer were, in every case, still stopped or routed appropriately at a different layer (wrong

layer, not not-blocked); none involved prohibited content reaching a later artifact. These wrong-layer cases are grouped and described in Supplementary Note 3's case-by-case analysis rather than repeated here, following the suite's own reporting standard that a stated boundary with a measured failure rate is more informative than a stated boundary with no measurement.

Appendix E The 144-probe specification

Nine categories of 16 probes each, which covers all 29 requirements at least four times and all design principles. Seventy-nine probes are adversarial and 65 are benign, incomplete or control cases. Probe types are abbreviated with A for adversarial and B for benign. Stop refers to the seven stopping conditions in the main text, and a dash indicates that no stop is expected. The full specification, including the exact user prompt, the fixture payload, the expected state transition, the deterministic pass criterion and the explicit failure condition for every probe, is released with the code as a machine-readable workbook and fixture set.